

Assignment 1

COL775: Deep Learning

Semester II, 2025–2026

Due Date: March 26, 2026

Notes:

- You should use Python and Pytorch for all your programming solutions.
 - You should submit all your code as well as any graphs that you might plot. Submit all the code in .py files and all answers to theory questions; all graphs should be submitted in a PDF. Do not submit any theory in the code files. Do not submit the datasets.
 - Include a **single write-up (pdf) file** which includes a brief description for each question explaining what you did. Include any observations and/or plots required by the question in this single write-up file.
 - Your code should have appropriate documentation for readability. Demo will also be conducted in which you will have to explain your code. You will be graded based on what you have submitted as well as your ability to explain your code.
 - The readability and presentation in your report will also carry marks. While there is no specific format, you are advised to be clear, concise and complete in your answers with explanations wherever required.
 - A substantial portion of your marks will be based on the accuracy achieved, so ensure that your models are trained thoroughly and properly.
 - Your assignment will be run on autograder scripts. Detailed instructions regarding this and submission guidelines of code will follow soon. Till then, please start working on writing the functions for your code.
 - Refer to the [course website](#) for assignment submission instructions.
 - We plan to run Moss on the submissions. We will also include submissions from previous years since some of the questions may be repeated. Any cheating will result in a zero on the assignment, an additional penalty upto the negative of the total weightage of the assignment and possibly much stricter penalties (including a **fail grade** and/or referring to a **DisCo**). Please note that using any **AI tools is not allowed**.
-

1 PART 1: ResNet and Normalization Schemes

Residual Networks (ResNet) [5] present a very simple idea to introduce identity mappings via residual connections. They are shown to significantly improve the quality of training (and generalization) in deeper networks. We covered the core idea in class. Before starting this part of the assignment, you should thoroughly read the ResNet paper. Specifically, we will implement the ResNet [5] architecture, and study the effect of different normalisation schemes, viz. Batch Normalization [6], Instance Normalization [11], Batch-Instance Normalization [11], Layer Normalization [1], and Group Normalization [12] within ResNet. We will experiment using a subset of ImageNet classification dataset.

1.1 Image Classification with ResNet-18 (Baseline)

We'll work with a 100-class subset of ImageNet-1k link is provided [here](#). It contains 680 training and 50 validation images per class of various resolutions.

1.1.1 Architecture

You must implement the ResNet-18 architecture **from scratch** by carefully reading the original paper [5]. *Do not use torchvision.models* or any pre-built ResNet implementation. The high-level structure of ResNet-18, including how spatial dimensions evolve for a 224×224 input, is summarized in Table 1. Use this as a *verification reference* after reading the paper — not as a substitute for it.

Table 1: ResNet-18 layer-by-layer dimensions for input $3 \times 224 \times 224$.

Stage	Operation	Stride	Output ($C \times H \times W$)
Input	—	—	$3 \times 224 \times 224$
Conv1	7×7 conv, 64 filters, pad 3	2	$64 \times 112 \times 112$
MaxPool	3×3 max pool, pad 1	2	$64 \times 56 \times 56$
Layer 1	Block 1 ($2 \times [3 \times 3$ conv, 64 ch])	1	$64 \times 56 \times 56$
	Block 2 ($2 \times [3 \times 3$ conv, 64 ch])	1	$64 \times 56 \times 56$
Layer 2	Block 1 ($2 \times [3 \times 3$ conv, 128 ch]) [†]	$2^\dagger$	$128 \times 28 \times 28$
	Block 2 ($2 \times [3 \times 3$ conv, 128 ch])	1	$128 \times 28 \times 28$
Layer 3	Block 1 ($2 \times [3 \times 3$ conv, 256 ch]) [†]	$2^\dagger$	$256 \times 14 \times 14$
	Block 2 ($2 \times [3 \times 3$ conv, 256 ch])	1	$256 \times 14 \times 14$
Layer 4	Block 1 ($2 \times [3 \times 3$ conv, 512 ch]) [†]	$2^\dagger$	$512 \times 7 \times 7$
	Block 2 ($2 \times [3 \times 3$ conv, 512 ch])	1	$512 \times 7 \times 7$
GAP	Global Average Pooling	—	$512 \times 1 \times 1$
FC	Linear	—	100

[†]Downsampling occurs in the *first* 3×3 convolution of this block only. The skip connection must be adjusted accordingly — refer to the paper for the correct projection strategy.

1.1.2 Data Preprocessing & Augmentation

- Resize and center-crop all images to exactly 224×224 before feeding them to the network.
- Apply at minimum: **Random Crop** and **Horizontal Flip** [7]. You may additionally explore CutOut [4], MixUp [13], or AutoAugment [3].

1.1.3 Training Setup

The recommended configuration is provided below. Minor deviations are permitted; however, the number of epochs must **not exceed** the prescribed value. You may increase the batch size (e.g., up to 256) if required. Any modifications must be applied *uniformly* across all experiments in Section 4.6.

- **Batch Size:** 128
- **Epochs:** 100 (maximum allowed)
- **Optimizer:** SGD with momentum 0.9 and weight decay 10^{-4}
- **Initial Learning Rate:** 0.05, with an appropriate scheduler.

You are encouraged to explore alternative optimizers (Adam, AdamW, RMSProp) and regularization strategies (Label Smoothing, Dropout). **Once finalized, the chosen setup must remain fixed for all reported experiments.**

Baseline: Train a standard **standard ResNet-18** model as a baseline using PyTorch’s built-in `nn.BatchNorm2d` placed after every convolution, exactly as described in the original paper [5]. This is the only place in the assignment where you are permitted to use a built-in normalization layer. All performance comparisons in Section 4.6 will be made against this baseline.

1.1.4 Analysis

Report the following statistics / analysis for all the models:

- Accuracy, Micro-F1, and Macro-F1 on Train and Validation splits.
- Training and Validation curves for: Loss, Accuracy, Micro-F1, and Macro-F1.

2 Impact of Normalization

The standard implementation of ResNet uses Batch Normalization [6]. In this part of the assignment, we will replace Batch Normalization with various other normalization schemes and study their impact.

2.1 Custom Implementations

Implement the following normalization schemes **from scratch** as subclasses of `torch.nn.Module`. Read each corresponding paper before implementing.

1. **No Normalization (NN)**
2. **Batch Normalization (BN)** [6]
3. **Instance Normalization (IN)** [11]
4. **Batch-Instance Normalization (BIN)** [8]
5. **Layer Normalization (LN)** [1]
6. **Group Normalization (GN)** [12]

2.2 Comparative Experiments

1. **Model Variants:** Starting from your baseline architecture (Section 1.1), create the following **6 variants** by modifying the normalization scheme. Normalization must be applied *immediately after each convolution layer* in all cases.

#	Variant	Description
1	NN	No normalization layers at all
2	BN	Custom Batch Normalization (from scratch)
3	IN	Custom Instance Normalization (from scratch)
4	BIN	Custom Batch-Instance Normalization (from scratch)
5	LN	Custom Layer Normalization (from scratch)
6	GN	Custom Group Normalization (from scratch)

Together with the baseline, you will train and evaluate **7 models** in total.

2. **Sanity Check:** Compare the error curves and performance statistics of the baseline model (Section 1.1) with the BN variant trained in this section. They should be (almost) identical.
3. **Training & Evaluation:** Train all 6 variants using the exact same configuration finalized in Section 1.1. Compare error curves and performance statistics (Accuracy, Micro-F1, Macro-F1) on Train, Validation splits across all 7 models.
4. **Batch Size Sensitivity (BN vs. GN):** Group Normalization is often cited as robust to small batch sizes, whereas Batch Normalization is known to degrade. Verify this empirically:
 - Retrain your BN and GN variants with a batch size of **8**.
 - Compare performance against your standard batch size (128) runs.
 - Note that reducing the batch size will significantly increase the training time. To reduce the time taken, run it for 50 epochs and early stop based on validation accuracy.

2.3 Visualizing Model Explanations via Grad-CAM

An important part of any model training is analysing why a model predicts any particular label for a given input. In this sub-part we'll use Grad-CAM [9] for producing visual-explanations from our ResNet models. Grad-CAM computes the gradient of input feature map with respect to the specified output class. Higher the gradient value at particular position indicates it's importance for predicting particular class. For more information please read Grad-CAM paper[9]. For this assignment use the Grad-CAM's implementation from this [package](#).

1. **Selection:** Choose **5 distinct classes** from the provided ImageNet-100.
2. **Visualization:** Using the *best-performing* model from all your variants, generate gradient activation maps (superimposed on the original image) for each of the 5 chosen classes:
 - **5 correctly** classified images per class.
 - **5 incorrectly** classified images per class.
 - Gradients should be computed with respect to the *ground-truth* class label.
3. **Analysis:** Discuss what features the model attends to. Do correct and incorrect predictions differ in their activation patterns? Are there consistent failure modes?

3 Submission Guidelines

Submit a single compressed folder named `EntryNumber_A1.zip` containing the following.

1. All Python scripts containing the training code.
2. A concise PDF report containing:
 - Brief description of your ResNet-18 implementation and any design choices made.
 - Loss and Accuracy plots (Train and Validation) for all 7 models.
 - Final Test Accuracy, Micro-F1, and Macro-F1 for all 7 models (baseline + 6 variants).
 - Discussion of performance differences across all normalization schemes.
 - Activation maps for all 5 classes (correct and incorrect predictions), with analysis
 - A publicly accessible Google Drive or OneDrive link to your .pth files. Do not upload model files directly to the submission portal.
3. We'll evaluate your trained models on the held-out test set on our end. Further instructions on this will be posted on piazza.

4 PART 2: Neural Machine Translation and Cross-Lingual Transfer

In this part of the assignment, you will implement and evaluate a neural machine translation system. We will implement a seq-2-seq architecture, implemented as a combination of an encoder (with a number of variants), and an LSTM based decoder with attention. In terms of the specific task, you will work on the task of translating **English** sentences into **Hindi**. Subsequently, you will investigate whether the knowledge learned by the English-Hindi translation model can be transferred to **Marathi**, which is closely related to Hindi, and also has the same script.

4.1 Dataset

[Dataset link](#).

The dataset folder contains the train, dev, and test files for both the English-Hindi and English-Marathi translation tasks. For the English-Hindi dataset, the train split contains 58494 samples, the dev and test set contains 3242 and 2256 samples respectively. Similarly, the English-Marathi dataset contains 18979 train samples, 1055 development samples, and 556 test samples. Additionally, we also plan to evaluate your English-Hindi checkpoint on a hidden set. A sample of the same has been provided for your reference.

4.2 Metrics

For the translation task, we will use the following metrics:-

- **BLEU Score** : The BLEU score measures the similarity between the generated translation and the reference translation using n-gram overlap.
- **chrF Score** : The chrF metric evaluates translation quality based on character n-gram overlap.
- **Translation Edit Rate (TER)** : This score measures the number of edits (insertions, deletions, substitutions, and shifts) required to turn a machine-generated translation into a human-referenced translation

There are many implementations of these metrics; to ensure fair comparison, use the implementation provided by **sacrebleu**. Below is a sample use case:

```
pip install sacrebleu
```

```
1 import sacrebleu
2
3 # Predictions and GTs are contained in an ordered list.
4 model_predictions = ['a', 'bb', 'c', ...]
5 ground_truths     = ['aa', 'bb', 'c', ...]
6
7 bleu = sacrebleu.corpus_bleu(model_predictions, [ground_truths]).score
8 chrF = sacrebleu.corpus_chrf(model_predictions, [ground_truths]).score
9 ter  = sacrebleu.corpus_ter(model_predictions, [ground_truths]).score
10
11 #Note that ground_truths is passed as a list-of-list,
12 #which is what the package expects to calculate correct metrics.
```

4.3 Models

For your experiments you are supposed to train the following models.

1. A Seq2Seq model with GloVe embeddings, using an Bi-LSTM encoder and an LSTM decoder. For details of an LSTM refer [10].

2. A Seq2Seq+Attention model with GloVe embeddings, using an Bi-LSTM encoder and an LSTM decoder.
3. A Seq2Seq+Attention model using a pre-trained frozen BERT-base-cased encoder and an LSTM decoder.
4. A Seq2Seq+Attention model with pre-trained BERT-base-cased encoder and an LSTM decoder, where the BERT encoder can now be fine-tuned along with the remaining network. (Save your model checkpoint, you'll use it in other experiment as well)

Your training configuration should be applied consistently across all experiments. You must report the following for each model:

- Loss curves on the training and dev set.
- BLEU, chrF and TER score curves on dev set.
- Final metric scores on the test set.
- All hyper-parameter settings used in your experiments.
- Any findings observed from any potential grid search performed.
- Sample translations generated by your model.

In this part, you will implement the models with **teacher forcing** with a ratio of **0.6** during training i.e for a given batch, teacher forcing is applied with 0.6 probability and model's predictions are used with 0.4 probability. You must set it to 0 during inference/validation.

4.4 Decoding using Beam Search

In this problem, during inference, the translation model generates the target sentence token by token, selecting the most probable token at each time step, this is called as Greedy Decoding. Alternatively, **Beam search** decoding maintains multiple candidate sequences during generation at each step and explores a larger search space to find better translations. You are required to implement **beam search decoding from scratch** in Python/PyTorch. During inference, generate translations using beam search and compare the results with those obtained using greedy decoding.

For the fourth model (Seq2Seq+Attention with fine-tuned Bert), generate translations for the test dataset using beam search with beam sizes {5, 10, 20}. Report all the scores on the test dataset for each beam size. Also, report the average decoding time per sentence. Analyze how the beam size influences the quality of the generated translations and comment on your observations. How do these numbers compare with the case of greedy decoding (Beam size = 1).

4.5 Effect of Teacher Forcing Probability

Train the fourth model (Seq2Seq+Attention with fine-tuned BERT) using a **teacher forcing schedule** rather than a fixed probability. Use an **inverse sigmoid decay schedule**[2], defined as:

$$p(t) = \frac{k}{k + \exp(t/k)} \quad (1)$$

where t is the current epoch and k is a decay constant (set $k = 8$). This schedule starts close to $p = 1$ (full teacher forcing) in early epochs and smoothly decays toward $p \approx 0$ by the end of training, giving the model strong guidance early on while forcing it to rely on its own predictions as it matures.

Plot the training loss and validation loss against epochs on two separate plots (one for training loss, one for validation loss), each also overlaying the teacher forcing probability curve on a secondary y -axis, so the relationship between the schedule and loss dynamics is visible. Report all scores on the test dataset for this configuration. Comment on how the decaying teacher forcing schedule affects translation quality and training stability compared to a fixed teacher forcing probability.

4.6 Cross-Lingual Transferability

In this section, we investigate whether the knowledge learned by the English–Hindi translation model can be transferred to another related Indic language, **Marathi**. Since Hindi and Marathi share linguistic similarities in terms of vocabulary, grammar, and sentence structure, the representations learned while translating English to Hindi may be useful for improving English to Marathi translation.

To study this effect, train a translation model on the **English–Hindi** dataset (You can use checkpoints trained in the previous section). Load your trained model and now train it on the **English–Marathi** dataset. For comparison, also train a baseline model for **English–Marathi** translation from scratch using the same architecture and training configuration. To ensure a fair comparison, both the baseline model and the transferred model should be trained on the English–Marathi dataset for the same number of epochs.

- Report the BLEU, chrF and TER scores for both the models i.e trained from scratch and transferred from English-Hindi on test set.
- Compare the results of both approaches and discuss whether cross-lingual transfer improves translation performance.
- Provide sample translations generated by both models and comment on qualitative differences.

4.7 Preprocessing

Make sure to perform the necessary preprocessing on the dataset, as it can significantly affect the translation performance. Proper preprocessing helps reduce noise in the data and improves the quality of the learned representations. For tokenization and embeddings, different strategies are used depending on the model architecture.

Encoder:

For the architectures that use GloVe embeddings, the source sentences should be tokenized using word-level tokenization, and the encoder embeddings should be initialized with the pre-trained **100-dimensional GloVe vectors**. For the architectures that use BERT, the English input sentences should be tokenized using the BERT tokenizer.

Decoder:

The decoder requires its own tokenizer for the target language. Therefore, the translated sentences should be tokenized using a **custom Byte Pair Encoding (BPE) tokenizer** trained on the target-side corpus. Below is the code for learning the tokenizer using `transformers` library.

```
pip install transformers
```

```
1 from tokenizers import Tokenizer
2 from tokenizers.models import BPE
3 from tokenizers.trainers import BpeTrainer
4 from tokenizers.pre_tokenizers import Whitespace
5 from transformers import PreTrainedTokenizerFast
6
7 # 1. Initialize an empty BPE Tokenizer
8 tokenizer = Tokenizer(BPE(unk_token="<unk>"))
9 tokenizer.pre_tokenizer = Whitespace()
10
11 # 2. Setup the trainer with your specific vocab size and special tokens
12 trainer = BpeTrainer(special_tokens=["<pad>", "<unk>", "<sos>", "<eos>"], vocab_size=5000)
13 # We've used arbitrary vocab_size here, you must set it carefully.
14
15 # 3. Train the tokenizer on your dataset
16 tokenizer.train_from_iterator(train_df["translated"].astype(str).tolist(), trainer)
17
18 # 4. Wrap it in HuggingFace's Fast framework for easy use in PyTorch
```

```

19 fast_tokenizer = PreTrainedTokenizerFast(tokenizer_object=tokenizer,
20                                         pad_token="<pad>", unk_token="<unk>",
21                                         bos_token="<sos>", eos_token="<eos>")
22
23 # 6. Save it to disk
24 fast_tokenizer.save_pretrained(save_dir)
25
26 # -----
27 # You can now load the saved tokenizer as below:
28 tgt_tokenizer = PreTrainedTokenizerFast.from_pretrained(save_dir)
29 example_text = "<hindi_text>"
30 encoded_ids = tgt_tokenizer(example_text)['input_ids']
31 # -----

```

5 Submission Guidelines

Submit a single compressed folder named `EntryNumber_A1.zip` containing the following.

1. All Python scripts containing the training code.
2. A concise PDF report containing:
 - Brief description of your models implementation and any design choices made.
 - train and Validation loss curves.
 - BLUE score and chrF scores.
 - Discussion of performance differences across all teacher forcing ratios and beam size.
 - A publicly accessible Google Drive or OneDrive link to your .pth files. Do not upload model files directly to the submission portal.
3. We'll evaluate your trained models on the held-out test set on our end. Further instructions on this will be posted on piazza.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
- [2] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks, 2015.
- [3] Ekin D. Cubuk, Barret Zoph, Dandelion Mané, Vijay Vasudevan, and Quoc V. Le. Autoaugment: Learning augmentation strategies from data. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 113–123, 2019.
- [4] Terrance DeVries and Graham W. Taylor. Improved regularization of convolutional neural networks with cutout, 2017.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [6] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.
- [7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Commun. ACM*, 60(6):84–90, May 2017.
- [8] Hyeonseob Nam and Hyo-Eun Kim. Batch-instance normalization for adaptively style-invariant neural networks, 2019.
- [9] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 128(2):336–359, October 2019.
- [10] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks, 2014.
- [11] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization, 2017.
- [12] Yuxin Wu and Kaiming He. Group normalization, 2018.
- [13] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization, 2018.